Project Overview

For our project, we built a basketball management system. It's a database that tracks the information you'd want to know about a basketball league and ties it together so you can pull useful results out of it. The goal was to model how a real league operates. Teams play games at stadiums, rosters of players and coaches are under contract, and stats accumulate over the course of a season. We wanted one system that holds all of that and keeps it consistent as new games are played and new results come in.

We also wanted the database to do more than just store information, so we set it up to handle calculations on its own. We built views that produce the common reports you'd want to see, like standings and season averages, so the work isn't repeated every time. We also wrote stored procedures for the operations that get run most often, like recording a game result or logging a player's stat line, so those updates happen in a single call and the data stays consistent.We loaded in sample data with real teams and players so we could test the system and demonstrate how it works.

ER Diagram

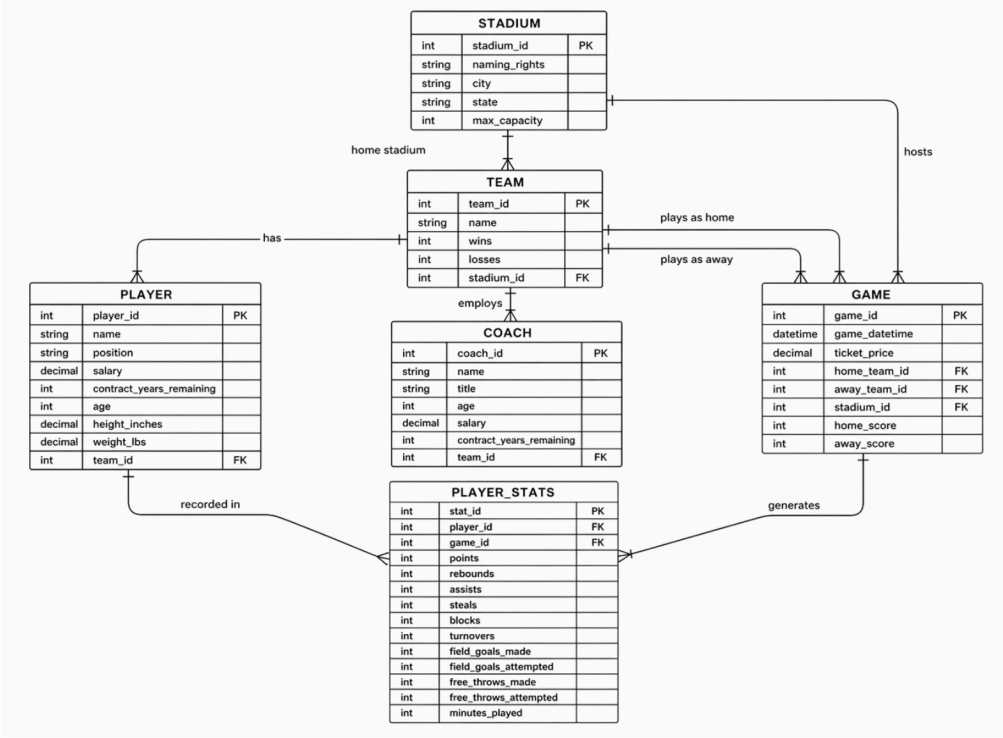

Figure 1. *ER Diagram*

Figure 1 shows the entities in our system and how they connect, but it doesn't explicitly label the cardinality of each relationship. This section walks through each relationship and clarifies how the entities relate to one another.

Stadium and Team (home stadium): One stadium serves as the home venue for one team, while each team has exactly one home stadium. This is enforced through the stadium_id foreign key on the Team entity.

Stadium and Game (hosts): A stadium hosts many games over the course of a season, while each game is played at exactly one stadium. This is a one-to-many relationship from Stadium to Game.

Team and Player (has): A team has many players on its roster, while each player belongs to exactly one team. This one-to-many relationship is enforced through the team_id foreign key on Player, which cannot be null.

Team and Coach (employs): A team employs multiple coaches, including a head coach and assistant coaches, while each coach works for exactly one team. This is also a one-to-many relationship.

Team and Game (plays as away) (plays as home): Every game involves two teams, one home and one away. Conceptually, this is a many-to-many relationship between Team and Game, which is resolved by storing both home_team_id and away_team_id as foreign keys directly on the Game entity. Each team can appear in many games, but every game has exactly one home team and one away team.

Player, Game, and Player Stats (recorded in) (generates): A player participates in many games, and a game involves many players, which makes this a many-to-many relationship. We resolve it through the Player_Stats entity, holding the stat line for each player in each game. The unique constraint on the combination of player_id and game_id ensures that a player can only have one stat line per game. Player_Stats has a one-to-many relationship with both Player and Game.

It's also worth noting that the ER diagram does not display unique constraints beyond primary keys. These are defined at the table level in our schema rather than in the diagram itself. For example, Team enforces a unique constraint on the team name, and Player_Stats enforces a unique constraint on the combination of player_id and game_id so that a player can only have one stat line per game. These constraints are documented in the table definitions section of the report.

Description of Tables and Relationships

The database is organized into six tables that together capture the structure of a basketball league. Each table is described below, along with its key attributes and how it connects to the rest of the schema.

Stadium: Stores venue information, including naming rights, city, state, and maximum capacity. Each stadium is identified by a stadium_id and serves as the home venue for one team and the location for many games.

Team: Stores the team name along with its current win and loss totals. Each team references a stadium through stadium_id and is the parent entity for its players, coaches, and games. The team name is enforced as unique.

Player: Stores roster information, including name, position, salary, contract years remaining, age, height, and weight. Position is restricted to the five standard basketball positions through an enumerated type. Each player is tied to a team through team_id.

Coach: Stores coaching staff information,n including name, title, age, salary, and contract years remaining. The title is restricted to either Head Coach or Assistant Coach. Each coach is tied to a team through team_id.

Game: Stores scheduled and completed games, including the date and time, ticket price, home and away teams, stadium, and final scores. Scores are nullable so that scheduled games can exist before they have been played. Each game references two teams and one stadium.

Player_Stats: Stores the stat line for each player in each game, including minutes played, points, rebounds, assists, steals, blocks, turnovers, and shooting numbers. It serves as the junction table between Player and Game, with a unique constraint on the combination of player_id and game_id to ensure one stat line per player per game.

The relationships between these tables are enforced through foreign keys with rules for what happens when records are updated or deleted. Most relationships use ON UPDATE CASCADE and ON DELETE RESTRICT, so updates flow through, but deletions are blocked if related records still exist, which prevents accidental loss of data. Player_Stats is the exception, using ON DELETE CASCADE so stat lines are removed automatically if their associated player is deleted. Check constraints are also applied throughout to enforce valid ranges, such as non-negative scores, salaries, and stat values, along with logical constraints like field goals made not exceeding field goals attempted.

Explanation of Implemented Operations

Beyond storing data, the database includes a set of operations that handle common tasks and reporting needs. These are implemented as views and stored procedures.

Views: Three views are defined to handle the queries that would otherwise need to be rewritten each time they are run. vw_standings returns the league standings by joining Team with Stadium and calculating win percentage from each team's wins and losses. vw_schedule returns the full game schedule with home team, away team, venue, ticket price, and a result column that displays the winning team if the game has been played or "Scheduled" if it has not. vw_player_season_avg returns season averages for every player, including points, rebounds, assists, steals, blocks, turnovers, and minutes per game, along with field goal and free throw percentages calculated across all games played.

Stored Procedures: Two stored procedures handle the operations that are performed most often. sp_record_game_result takes a game ID and the final scores for the home and away teams, updates the game record with those scores, and increments the wins and losses of the appropriate teams based on the outcome. This keeps the team standings consistent without requiring multiple manual updates. sp_upsert_player_stats takes a player ID, game ID, and a full stat line, then either inserts a new row into Player_Stats or updates the existing row if one already exists for that player and game. This is handled through an ON DUPLICATE KEY UPDATE clause that relies on the unique constraint discussed earlier.

Together, these operations reduce repetitive query writing and ensure that updates affecting multiple tables happen in a single call, which keeps the data consistent across the system.

Challenges Faced During the Project

games, since each game involves two teams in different roles. We resolved this by including both home_team_id and away_team_id as separate foreign keys on the Game table. We also spent time working through what should happen on each foreign key when records are updated or deleted, making sure those rules behaved correctly without leaving orphaned records or wiping out data we wanted to keep. Coordinating the schema design across the team also required a few rounds of revision before everyone agreed on the final structure.

Discussion

While the current system covers the core functionality of a basketball management database,

there are several directions it could be extended in the future.

The most obvious addition would be a frontend application that lets users interact with the database without writing SQL. A web interface could display standings, schedules, and stats in a clean format and allow staff to record results or update rosters through forms instead of stored procedure calls.

The schema itself could also be expanded to track more of what a real league deals with, like injuries, trades, draft picks, multiple seasons of historical data, and ticket sales. Additional views could support things like head-to-head matchups or advanced statistics such as player efficiency rating and true shooting percentage. Role-based access for coaches, analysts, and front office staff would also be a natural next step.

AI Usage Declaration

In this project, we have utilized ChatGPT in several aspects, such as structuring the ER diagram, improving the database design, and assisting in SQL query development. AI acted as a supporting source and was employed in enhancing comprehension and detecting mistakes. However, all the project-related decisions remained at our discretion.